



JUSKO Nathaniel (Chef de Projet) , DESSENEUX-AURIOL
Florient, MAKHLOUF Zakaria, BELHAMIDA Adil, BOUSLIM
Mourad



RAPPORT

SAE 2.02

***Comparaison algorithmique
d'un problème***

Sommaire:

Répartition du travail :	2
Modélisation du livre-jeu par graphe	3
Attributs des sommets	3
Objets requis :	3
Solution	3
Mesure de performance	4
Diagramme de classe Première version :	4
1. Présentation de l'application	5
1.1 Architecture du projet et évolutions du diagramme de classes	5
2. Justification de la correction des algorithmes	7
2.1 Algorithme BFS	7
2.2 Algorithme de Dijkstra	7
3. Évaluation expérimentale	8
3.1 Protocole	8
3.2 Comparaison BFS vs Dijkstra	8
3.3 Comparaison GenerateurV1 vs GenerateurV2	9
4. Manuel utilisateur	9
4.1 Prérequis	9
Pour vérifier l'installation :	9
4.2 Compilation avec Maven	10
Compiler et générer le JAR :	10
4.3 Exécution	10
4.4 Menu de l'application	10
4.5 Format du fichier CSV	12
Conclusion	12

Répartition du travail :

Notre groupe est initialement composé de quatre membres. En cours de projet, Mourad Bouslim nous a rejoints suite à des circonstances particulières : les membres de son groupe d'origine ont tous rencontré des difficultés personnelles les amenant à envisager de quitter l'IUT, ce qui a conduit M. Lechoppier à l'intégrer à notre équipe. Malgré cette arrivée en cours de route, Mourad a rapidement pris en charge des tâches importantes et s'est bien intégré au groupe.

Nathaniel a assuré le rôle de chef de projet. Il a créé le dépôt GitHub et le document partagé, organisé le travail du groupe, corrigé les problèmes liés à Git au fil du projet, participé à la correction de la modélisation du livre-jeu avec les graphes, et développé les tests ainsi que la classe Objet.

Florient a pris en charge la modélisation du livre-jeu avec les graphes en début de projet, puis a développé les tests et la classe Page.

Adil a travaillé sur la création du diagramme de classes, ainsi que sur les tests et la classe Enigme.

Zakaria MAKHLOUF a également participé à la création du diagramme de classes. Il a de plus implémenté les algorithmes de recherche (BFS et Dijkstra), développé la classe LivreJeu avec ses tests, et mis en place la classe AppLivreJeu qui constitue le point d'entrée de l'application.

Mourad BOUSLIM, arrivé en cours de projet, a pris en charge l'implémentation de GenerateurV2 (génération interactive du livre-jeu) ainsi que les tests de la classe Solution.

Modélisation du livre-jeu par graphe

On modélise le livre-jeu par un graphe orienté $G = (V, E)$ où :

- V est l'ensemble des sommets, chacun représentant une page du livre.
- $E \subseteq V \times V$ est l'ensemble des arêtes orientées : une arête $(u, v) \in E$ signifie que depuis la page u , le joueur peut aller à la page v .

Deux sommets sont distingués :

- $s \in V$: la page de départ
- $t \in V$: la page d'arrivée (fin du jeu)

Attributs des sommets

Chaque sommet $v \in V$ porte :

- $c(v) \in \mathbb{R}$: le temps estimé pour résoudre l'énigme de la page v
- $\text{obj}(v) \subseteq O$: les objets récupérables sur la page v , où O est l'ensemble de tous les objets du jeu

Objets requis :

On note $R \subseteq O$ l'ensemble des objets nécessaires pour terminer le jeu.

Solution

Une solution est un chemin dans G :

$$P = (v_0, v_1, \dots, v_k)$$

tel que :

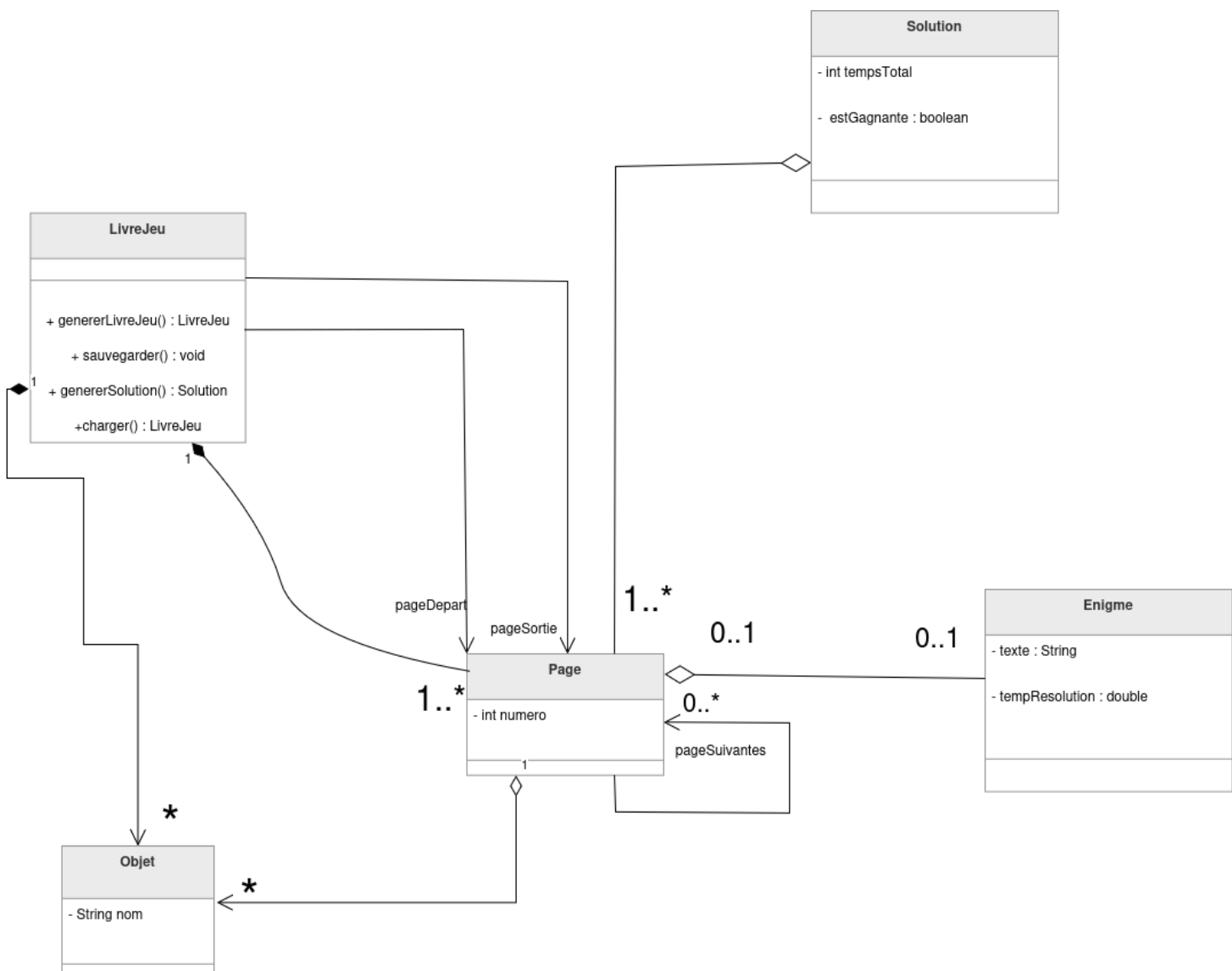
1. $v_0 = s$ (départ)
2. $v_k = t$ (arrivée)
3. $(v_i, v_{i+1}) \in E$ pour tout $i \in \{0, \dots, k-1\}$
4. $R \subseteq \bigcup \text{obj}(v_i)$ pour i de 0 à k (tous les objets requis sont collectés)

Mesure de performance

La longueur d'une solution est le nombre de pages visitées :

$$L(P) = k + 1$$

Diagramme de classe Première version :



1. Présentation de l'application

L'application qu'on a développée en Java permet de modéliser un livre-jeu sous forme de graphe orienté pondéré. Concrètement, on peut :

- charger un livre depuis un fichier CSV
- générer un nouveau livre automatiquement ou en mode interactif
- rechercher une solution avec BFS ou Dijkstra
- sauvegarder le livre généré

Le graphe utilisé est un `SimpleDirectedWeightedGraph` de la bibliothèque `JGraphT`. Chaque sommet est une `Page`, chaque arc est un `DefaultWeightedEdge`, et le poids d'un arc correspond au temps de résolution de l'énigme de la page source.

1.1 Architecture du projet et évolutions du diagramme de classes

Au départ, le diagramme de classes était assez basique : on avait les classes métier (`Page`, `Enigme`, `Objet`, `LivreJeu`, `Solution`) et c'est à peu près tout. Au fur et à mesure du développement, on a été amenés à l'étoffer pas mal.

La première grosse évolution a été d'ajouter l'interface `AlgorithmeRecherche`, qui définit une méthode `resoudre(LivreJeu livre) : Solution`. Les classes `AlgorithmeDijkstra` et `AlgorithmeBFS` implémentent toutes les deux cette interface. L'avantage c'est qu'on peut changer d'algorithme dans `AppLivreJeu` sans toucher au reste du code c'est le principe du polymorphisme qu'on a vu en cours.

On a fait pareil pour la génération avec l'interface `IGenerateur`. Elle est implémentée par `GenerateurV1` (génération aléatoire) et `GenerateurV2` (génération interactive). Les deux exposent aussi une méthode `genererDepuisPages()` adaptée à leur fonctionnement.

On a aussi créé `AppLivreJeu` pour séparer proprement la logique d'interaction utilisateur du reste. C'est cette classe qui gère le menu, lit les saisies au clavier avec un `Scanner`, et orchestre les appels aux générateurs et aux algorithmes.

Pour éviter de surcharger `LivreJeu` avec des responsabilités qui ne lui appartiennent pas, on a isolé deux traitements dans des classes utilitaires : `LecteurHistoire` pour le chargement du CSV, et `SauvegardeLivreJeu` pour l'export du graphe au format DOT. Ces classes ne

contiennent que des méthodes statiques, ce qui correspond bien à leur rôle d'utilitaires sans état.

Enfin, on a ajouté equals() et hashCode() dans Page et Objet car sans ça, les Set et Map ne fonctionnaient pas correctement (par défaut Java compare les références, pas les valeurs).

Description des classes principales :

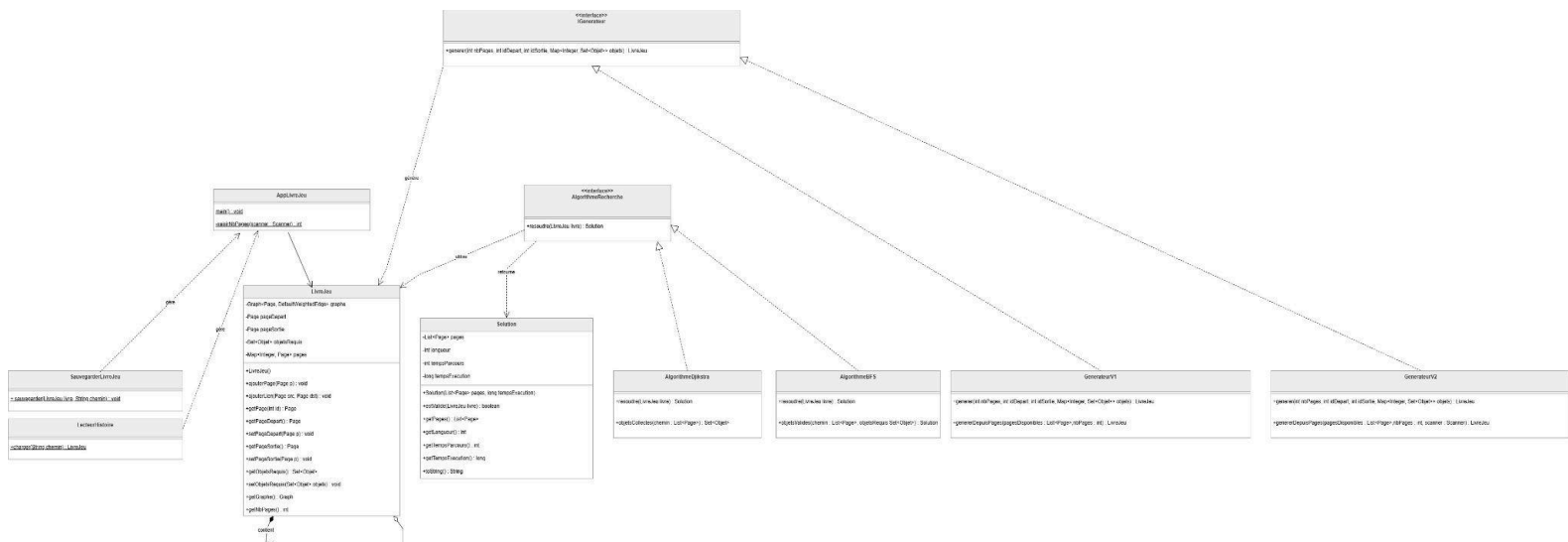
La classe LivreJeu est la structure centrale. Elle encapsule le graphe JGraphT, la page de départ, la page de sortie, la map des pages par identifiant, et les objets requis. C'est elle qu'on passe aux algorithmes pour qu'ils puissent travailler.

La classe Page modélise une page du livre. Elle a un identifiant entier, une Enigme, et un Set<Objet> pour les objets récupérables sur cette page. Les méthodes equals() et hashCode() sont basées uniquement sur l'identifiant.

La classe Enigme contient le texte de l'énigme et son temps de résolution. C'est ce tempsResolution qui sert de poids aux arcs du graphe.

La classe Objet est simple : juste un nom. Là aussi, equals() et hashCode() sont redéfinis sur le nom pour pouvoir comparer des objets par valeur.

La classe Solution stocke le chemin trouvé (une List<Page>), la longueur du chemin, le temps de parcours total (somme des temps d'énigmes), et le temps d'exécution de l'algorithme en millisecondes.



note : Voici notre dernière version de Diagramme de classe voir lien sur github

2. Justification de la correction des algorithmes

2.1 Algorithme BFS

BFS (Breadth-First Search) explore le graphe en largeur : on visite d'abord toutes les pages à 1 arc de distance, puis celles à 2 arcs, etc. On stocke dans une file (LinkedList) non pas des pages mais des chemins complets, ce qui nous permet de reconstruire directement la solution sans avoir à remonter des prédécesseurs.

Le truc un peu délicat avec le livre-jeu c'est la contrainte des objets. Deux passages par la même page peuvent mener à des états différents selon les objets qu'on a déjà collectés. Du coup, pour éviter les boucles infinies, on évite de revisiter une page déjà présente dans le chemin courant (!cheminCourant.contains(voisin)).

Terminaison : comme on n'ajoute jamais deux fois la même page dans un chemin, et que le nombre de pages est fini, la file ne peut pas grossir indéfiniment. L'algorithme se termine donc forcément.

Validité : quand on atteint la page de sortie, on vérifie que l'union des objets de toutes les pages du chemin contient bien tous les objets requis. Si ce n'est pas le cas, on continue l'exploration. La solution retournée satisfait donc bien les quatre conditions de validité.

Optimalité en longueur : BFS explore les chemins dans l'ordre croissant de leur longueur (d'abord les chemins de longueur 1, puis 2, etc.). Le premier chemin valide trouvé est donc forcément le plus court en nombre de pages. C'est exactement ce qu'on veut quand on minimise $L(P)$.

2.2 Algorithme de Dijkstra

Dijkstra calcule le chemin de coût minimum depuis un sommet source. Dans notre cas, le coût d'un arc correspond au tempsResolution de l'énigme de la page source, ce qui nous permet de minimiser le temps total de parcours.

On initialise le coût de la page de départ à 0 et celui de toutes les autres à Integer.MAX_VALUE (notre équivalent de l'infini). À chaque itération, on sélectionne la page non encore marquée ayant le coût minimal, on la marque, puis on tente de relaxer ses voisins.

Terminaison : à chaque tour de boucle, une page est marquée et retirée de la file de priorité. Comme il y a un nombre fini de pages, la boucle se termine après au plus $|V|$ itérations.

Validité : l'invariant classique de Dijkstra garantit que quand une page est marquée, son coût est définitif et minimal. Le chemin reconstruit en remontant le tableau predecesseur est donc un chemin valide de la source à la cible.

Optimalité en temps : Dijkstra trouve le chemin dont la somme des tempsResolution est minimale, ce qui correspond exactement à minimiser le temps de parcours tel que défini dans le sujet.

Limite de notre implémentation : notre Dijkstra ne gère pas finement la contrainte des objets. Il calcule le chemin le plus rapide, puis vérifie en une seule fois si ce chemin collecte tous les objets requis. Si ce n'est pas le cas, il retourne null sans chercher d'alternative. En pratique sur nos livres générés, les objets requis correspondent à l'ensemble de tous les objets disponibles, donc tout chemin qui passe par toutes les pages collecte automatiquement toute la limite ne se voit pas sur nos tests.

3. Évaluation expérimentale

3.1 Protocole

Le temps d'exécution est mesuré directement en Java avec `System.currentTimeMillis()` juste avant et juste après l'appel à `resoudre()`. La différence est stockée dans l'objet `Solution` et affichée dans le `toString()`. Les tests ont été réalisés sur des livres générés par `GenerateurV1` depuis notre fichier `histoire.csv`.

3.2 Comparaison BFS vs Dijkstra

Sur nos tests, les deux algorithmes trouvent toujours une solution quand il en existe une. Les résultats obtenus sont les suivants :

Nb pages	Algorithme	Longueur solution	Temps parcours (s)	Temps exécution (ms)
10	BFS	9 pages	72	10
10	Dijkstra	9 pages	72	2

Note : le tableau mériterait d'être complété avec des tests sur des livres plus grands (20, 50, 100 pages). Sur nos tests à 10 pages, les deux algorithmes trouvent la même solution, ce qui s'explique par la structure linéaire du générateur V1. Dijkstra est nettement plus rapide à l'exécution (2 ms vs 10 ms) car il ne stocke pas de chemins complets en mémoire comme BFS.

Ce qu'on retient : BFS garantit le chemin le plus court en nombre de pages, mais au prix d'une exploration plus coûteuse en mémoire. Dijkstra est plus efficace à l'exécution et optimise le temps de parcours, mais peut rater des solutions si la contrainte objets n'est pas satisfaite par le chemin optimal. Pour des livres de petite taille (< 200 pages), les deux approches sont tout à fait utilisables.

3.3 Comparaison GenerateurV1 vs GenerateurV2

GenerateurV1 sélectionne aléatoirement n pages dans le CSV, les relie en chaîne linéaire et ajoute des liens retour vers le départ tous les 3 pas. Aucune interaction utilisateur n'est nécessaire, ce qui le rend utilisable pour des tests automatisés.

GenerateurV2 demande à l'utilisateur de choisir manuellement les pages, les liens et les objets requis via la console. Cette approche donne plus de contrôle sur la structure du livre, mais elle n'est pas adaptée aux tests de performance automatisés puisqu'elle nécessite des saisies à chaque étape.

En termes de temps d'exécution pur, GenerateurV1 est forcément plus rapide car il n'attend pas de saisie utilisateur. GenerateurV2 dépend entièrement de la vitesse de frappe de l'utilisateur et ne peut donc pas être comparé sur ce critère de façon significative.

4. Manuel utilisateur

4.1 Prérequis

Pour compiler et exécuter l'application, il faut avoir installé :

1. **Java JDK 17 ou supérieur**
2. **Maven 3.8 ou supérieur**

Pour vérifier l'installation :

-
- **bash**
 - **java -version**
 - **mvn -version**

4.2 Compilation avec Maven

Se placer dans le dossier du projet (celui qui contient pom.xml) :

1. **bash**
2. **cd livrejeu**

Compiler et générer le JAR :

1. **bash**
2. **mvn clean package -DskipTests**
3. **Si tout se passe bien, le JAR est généré dans target/ :**
 - **target/livrejeu-1.0-SNAPSHOT.jar**

4.3 Exécution

il faut obligatoirement lancer la commande depuis livrejeu/ et non depuis target/, car l'application charge histoire.csv avec un chemin relatif.

4.4 Menu de l'application

Au démarrage, l'application charge automatiquement le fichier histoire.csv puis affiche un menu texte :

MENU

1. Generer un livre (aleatoire - V1)
2. Generer un livre (interactif - V2)
3. Afficher le livre
4. Resoudre

5. Sauvegarder (CSV)

0. Quitter

L'application propose un menu interactif en ligne de commande composé de six options, accessibles à tout moment durant l'exécution du programme.

Option 1 : Génération aléatoire (V1) : l'utilisateur entre uniquement le nombre de pages souhaité. Le programme sélectionne automatiquement les pages, crée les liens entre elles et définit une page de départ et une page de sortie de façon aléatoire à partir du fichier `histoire.csv`.

Option 2 : Génération interactive (V2) : cette option guide l'utilisateur étape par étape. Il choisit lui-même les pages par leur identifiant, définit manuellement les liens entre elles, puis sélectionne les objets requis. Seuls les objets présents dans les pages choisies sont proposés, ce qui évite toute incohérence.

Option 3 : Affichage du livre : affiche le livre actuellement en mémoire (nombre de pages, page de départ, page de sortie). Utile pour vérifier la structure avant de lancer la résolution.

Option 4 : Résolution : l'utilisateur choisit entre Dijkstra (chemin le plus rapide en temps de résolution) et BFS (chemin le plus court en nombre de pages). La solution est affichée avec le détail des pages, le temps de parcours et le temps d'exécution.

Option 5 : Sauvegarde : exporte le livre en cours vers un fichier `.dot` permettant de visualiser le graphe avec Graphviz (`dot -Tpdf monlivre.dot -o monlivre.pdf`).

Option 0 : Quitter : ferme le scanner et termine l'application proprement.

Exemple de session :

Votre choix : 1

Nombre de pages : 10

OK : Livre généré avec 10 pages.

Votre choix : 4

1. Dijkstra — chemin le plus RAPIDE

2. BFS — chemin le plus COURT

Votre choix : 2

=== Solution ===

Longueur : 9 pages

Temps parcours : 18 sec

Temps execution: 2 ms

4.5 Format du fichier CSV

Le fichier `histoire.csv` se trouve dans `src/main/java/com/livrejeu/histoireCsv/`. Chaque ligne correspond à une page selon le format :

`texte_enigme,temps_resolution,objet1;objet2;...`

Exemple :

Vous êtes devant une porte verrouillée...,3,clé

Un garde vous barre le passage...,5,

Pour enrichir le livre, il suffit d'ajouter des lignes en respectant ce format. Les lignes sans objets doivent quand même avoir la virgule finale.

Conclusion

Ce projet nous a permis de mettre en pratique les notions de graphes vues en cours, notamment les algorithmes de parcours BFS et Dijkstra. On a réalisé assez vite que les deux algorithmes n'optimisent pas la même chose : BFS minimise le nombre de pages visitées, Dijkstra minimise le temps total de résolution des énigmes. Choisir le "meilleur" algorithme dépend donc de ce qu'on veut optimiser.

La partie la plus compliquée a été de gérer correctement la contrainte des objets requis dans BFS sans tomber dans des boucles infinies. Il a fallu réfléchir à comment représenter l'état de l'exploration pour ne pas revisiter deux fois la même configuration.

Sur le plan organisationnel, l'intégration de Mourad en cours de projet a été un défi supplémentaire, mais ça s'est bien passé grâce à Git qui nous a permis de travailler en parallèle sans trop de conflits.

L'IA a été utilisée uniquement pour la relecture orthographique et la mise en forme de certaines phrases. L'ensemble du contenu technique modélisation, code, algorithmes, analyse a été réalisé par le groupe.